

Notes on Dantzig-Wolfe Decomposition

1. Block-Angular Structure

A matrix A has a **block angular structure** if it is of this form:

$$A = \begin{bmatrix} A_1 & A_2 & \cdots & A_{n-1} & A_n \\ A_{n+1} & 0 & \cdots & 0 & 0 \\ 0 & A_{n+2} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & A_{2n} \end{bmatrix}$$

where A_i are submatrices of size $m_i \times n_i$. We call the first n matrices $A_1, A_2, \dots, A_{n-1}$ the bounding constraints and the last n matrices the independent constraints. Notice that this easily becomes a linear program that looks like this,

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n \vec{c}_i \vec{x}_i \\ & \text{subject to} && [A_1, \dots, A_n, I] \vec{x}^T = \vec{b}_0 \quad \text{linking constraints} \\ & && A_{n+i} \vec{x}_i = \vec{b}_i, \quad \forall i \in \{1, \dots, n\} \quad \text{independent constraints} \\ & && \vec{x}^T \geq 0, \quad \vec{x}_i \geq 0, \quad \forall i \in \{1, \dots, n\} \quad \text{non-negativity constraints} \end{aligned}$$

In this linear program, the $\vec{c}_i$ are the costs and $\vec{x} = [\vec{x}_1, \dots, \vec{x}_n]^T$ are the decision variables and $\vec{b}_i$ are the right hand side of the constraints. The aim of the Dantzig-Wolfe decomposition is to be more computationally efficient in solving this problem. We start with a **restricted master problem** (RMP), which usually encompasses all the linking constraints and then check the independent constraints one by one by solving them as subproblems.

2. Column Generation

2.1 Intro

Column generation is a method for solving large scale linear programs. It is a variant of the simplex algorithm that is used to solve linear programs with a large number of variables. The idea is to start with a small subset of the variables and then iteratively add variables to the problem until the solution is optimal.

In a more formal sense, we have our **master problem** (MP) and our **restricted master problem** (RMP). The RMP is obtained by taking a subset of the variables from the MP. We iteratively solve the RMP and then add variables to the RMP that were not in the optimal solution. This process is repeated until the MP is optimal.

3. Dantzig-Wolfe Decomposition

3.1 Intro

We'll start by giving a high level view of the algorithm, which is basically doing these steps (Wikipedia):

1. Start with a feasible solution to the RMP, formulate new objective functions for each subproblem such that the subproblems will offer solutions that improve the current objective of the master program.
2. Re-solve the subproblems given their new objective functions, the optimal value for each subproblem is offered to the master program.
3. Incorporate one or all of the new columns generated by the solutions to the subproblems into the master program based on their respective ability to improve the original problem's objective.
4. Perform x iterations of the simplex algorithm in the master program, where x is the number of columns incorporated.
5. If the objective is improved, go to step 1. Otherwise, we ran out of improvement so we return the solution.

Note that Dantzig-Wolfe is most effective when the subproblems are easy to solve. Now, let's break it down into more detail. The first step consists of solving the RMP and then defining the objective functions. The RMP looks like this,

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n \vec{c}_i \vec{x}_i \\ & \text{subject to} && [A_1, \dots, A_n, I] \vec{x}^T = \vec{b}_0 \\ & && \vec{x}^T \geq 0 \end{aligned}$$

The subproblems look like this, for every i th subproblem,

$$\begin{aligned} & \text{maximize} && \vec{c}_i \vec{x}_i \\ & \text{subject to} && A_{n+i} \vec{x}_i = \vec{b}_i \\ & && \vec{x}_i \geq 0 \end{aligned}$$

What interests us is the feasible region of the subproblems, we will denote them as $S_i = \{\vec{x}_i \geq 0 | A_{n+i}\vec{x}_i = \vec{b}_i\}$. Assume that S_i is bounded for simplicity (it can easily be extended to having unbounded regions, we would consider unbounded rays). We have a set of vertices $V_i = \{(v_{i,1}, \dots, v_{i,n}) | v_{i,j} \in S_i\}$ which completely defines the feasible region of the subproblem. This implies that we can represent every $x_i \in S_i$ as a convex combination of the vertices $v_{i,j}$, i.e. $\exists \{\lambda_{i,j} \geq 0\}_j$ s.t. $x_i = \sum_{j=1}^n \lambda_{i,j} v_{i,j}$ and with $\sum_{j=1}^n \lambda_{i,j} = 1$ (this follows from the representation theorem).

Now, we can transform the RMP by using the $\lambda_{i,j}$ as the new decision variables.

$$\begin{aligned} \min \quad & \sum_{i=1}^m \sum_{j=1}^{N_i} \lambda_{i,j} (c_i v_{i,j}) \\ \text{s.t.} \quad & \sum_{i=1}^m \sum_{j=1}^{N_i} \lambda_{i,j} (A_{0,i} v_{i,j}) = b_0 \\ & \sum_{j=1}^{N_i} \lambda_{i,j} = 1 \quad \forall i = 1, 2, \dots, m \\ & \lambda_{i,j} \geq 0 \quad \forall i, j \end{aligned}$$

Notably, we reduced the number of constraints to $m + m_0$, since there are now m independent constraints and m_0 linking constraints. Basically, we squashed all the independent constraint blocks into single constraints. However, there are now way more variables, which is why we will use an altered simplex method.

3.2 Altered Simplex

We start by creating the dual variables, $y_0 \in \mathbb{R}^{m_0}$ and $z \in \mathbb{R}^m$ which set the sum $\sum_{i=1}^m \lambda_{i,j} = 1$. Note that each column represents one vertex. And so, we want to find out which columns have negative reduced cost (because this is a minimization problem). If we consider the column for $\lambda_{i,j}$, then the reduced cost is

$$\bar{c}_i = c_i v_{i,j} - y_0 A_{0,i} v_{i,j} - z_i$$

which we can obtain by taking the inner product of the dual variables with the column for $\lambda_{i,j}$.

$$\begin{bmatrix} y_0 & z \end{bmatrix} \begin{bmatrix} A_{0,i} v_{i,j} \\ c_i \end{bmatrix} \quad (\text{unsure about this})$$

So now we want to know if there exists $v_{i,j}$ such that $\bar{c}_i < 0$ or equivalently, $c_i v_{i,j} - y_0 A_{0,i} v_{i,j} - z_i < 0$. If so, we can then find the column for $\lambda_{i,j}$ that

has the smallest reduced cost? This means we want to find a vertex of S_i such that $(c_i - y_0 A_{0,i})v_{i,j} - z_i < 0$.

We now consider the resulting subproblem,

$$\begin{array}{ll} \min & \bar{c}_i x_i \\ \text{s.t.} & A_{n+i} x_i = b_i, \\ & x_i \geq 0 \end{array}$$

Since S_i is nonempty (otherwise the master problem is infeasible), this problem must contain a vertex $\mathbf{v}$ which is the optimal solution. If $\bar{c}_i \mathbf{v} < z_i$ then we know that $\mathbf{v}$ is a negative cost vertex and we should add it to the basis. If this is false $\forall i$, then no column of S_i has negative reduced cost. Hence, after solving the subproblems for all S_i , we either find a negative reduced cost column or otherwise we show that the current solution to the master problem is optimal.

Notice that this in itself is fundamentally a revised version of the simplex algorithm, however with significantly reduced usage of space. Additionally, it is more efficient computationally since the slave problems can be solved in parallel (that is the big advantage), we simply choose the first negative reduced cost column subproblem to be computed.

3.3 Examples

Our particular interest with this algorithm stems from the particular structure of the optimization problem we are trying to solve. Let us formalize the problem of block allocation (mining blocks) in the context of mine planning. Our particular problem is of this form,

$$\begin{aligned}
& \max \sum_{b \in B} (V_{b,A,I} m_{b,A,I} + V_{b,A,II} m_{b,A,II} + V_{b,B,I} m_{b,B,I} + V_{b,B,II} m_{b,B,II}) \\
& \text{st. } \sum_{b \in BB} \left(\frac{m_{b,A,I}}{r_{A,I}} + \frac{m_{b,A,II}}{r_{A,II}} + \frac{m_{b,B,I}}{r_{B,I}} + \frac{m_{b,B,II}}{r_{B,II}} \right) \leq t_{\text{period}} \quad (1) \\
& r_{A,I} m_{b,A,I} + r_{A,II} m_{b,A,II} = 0 \quad \forall b \in \mathcal{B} \quad (2) \\
& r_{B,I} m_{b,B,I} + r_{B,II} m_{b,B,II} = 0 \quad \forall b \in \mathcal{B} \quad (3) \\
& m_{b,A,I} + m_{b,B,I} \leq m_{b,I} \\
& m_{b,A,I} + m_{b,B,I} \leq m_{b,I} \quad \forall b \in \mathcal{B} \quad (4) \\
& m_{ijk}, r_{jk} \geq 0 \quad \forall i \in \mathcal{B}, j \in \text{modes}, k \in \text{ores}
\end{aligned}$$

With (1) being that total processing time cannot be greater than the total period and (2) being that the total amount of ore being processed in a block cannot exceed the capacity of the block. It is to be noted that we are missing a constraint related to the rate of extraction.

Now, we will look at two examples to illustrate this, both with two modes (A, B) and two ore types (I, II), one with one block, and one with two blocks.

2.3.1 With one block, we have this problem,

$$\begin{aligned}
& \max (V_{A,I} m_{A,I} + V_{A,II} m_{A,II} + V_{B,I} m_{B,I} + V_{B,II} m_{B,II}) \\
& \text{st. } \frac{m_{A,I}}{r_{A,I}} + \frac{m_{A,II}}{r_{A,II}} + \frac{m_{B,I}}{r_{B,I}} + \frac{m_{B,II}}{r_{B,II}} \leq t_{\text{period}} \quad (1) \\
& m_{A,I} + m_{B,I} \leq m_I \\
& m_{A,I} + m_{B,I} \leq m_I \\
& m_{A,I}, m_{A,II}, m_{B,I}, m_{B,II}, r_{A,I}, r_{A,II}, r_{B,I}, r_{B,II} \geq 0
\end{aligned}$$

So, now our first step,

2.3.2 With two blocks, we have this problem,

$$\begin{aligned}
& \max V_A(m_{1,A,I} + m_{1,A,II} + m_{2,A,I} + m_{2,A,II}) + V_B(m_{1,B,I} + m_{1,B,II} + m_{2,B,I} + m_{2,B,II}) \\
& \text{st. } \frac{m_{1,A,I} + m_{2,A,I}}{r_{A,I}} + \frac{m_{1,A,II} + m_{2,A,II}}{r_{A,II}} + \frac{m_{1,B,I} + m_{2,B,I}}{r_{B,I}} + \frac{m_{1,B,II} + m_{2,B,II}}{r_{B,II}} \leq t_{\text{period}} \quad (1) \\
& (r_{1,A,I}m_{1,A,I} + r_{2,A,I}m_{2,A,I}) - (r_{1,A,II}m_{1,A,II} + r_{2,A,II}m_{2,A,II}) = 0 \\
& (r_{1,B,I}m_{1,B,I} + r_{2,B,I}m_{2,B,I}) - (r_{1,B,II}m_{1,B,II} + r_{2,B,II}m_{2,B,II}) = 0 \\
& m_{1,A,I} + m_{1,B,I} \leq m_{1,I} \\
& m_{1,A,II} + m_{1,B,II} \leq m_{1,II} \\
& m_{2,A,I} + m_{2,B,I} \leq m_{2,I} \\
& m_{2,A,II} + m_{2,B,II} \leq m_{2,II} \\
& m_{i,j,k}, r_{j,k} \geq 0 \quad \forall i \in \{1, 2\} \quad j \in \{A, B\} \quad k \in \{I, II\}
\end{aligned}$$

Then we the feasible polytope for the subproblems, we have

Firstly, we rewrite it in terms of subproblem formulation, as follows,

$$\begin{aligned}
& \max[V_A, V_B](x_{1,I}^T + x_{2,I}^T + x_{1,II}^T + x_{2,II}^T) \\
& \text{st. } \left[\frac{1}{r_{A,I}}, \frac{1}{r_{B,I}}\right](x_{1,I}^T + x_{2,I}^T) + \left[\frac{1}{r_{A,II}}, \frac{1}{r_{B,II}}\right](x_{1,II}^T + x_{2,II}^T) \leq t_{\text{period}} \quad (1) \\
& ([r_{A,I}, 0]x_{1,I}^T + [r_{A,I}, 0]x_{2,I}^T) - ([r_{A,II}, 0]x_{1,II}^T + [r_{A,II}, 0]x_{2,II}^T) = 0 \\
& ([0, r_{B,I}]x_{1,I}^T + [0, r_{B,I}]x_{2,I}^T) - ([0, r_{B,II}]x_{1,II}^T + [0, r_{B,II}]x_{2,II}^T) = 0 \\
& x_{b,i} \in X_{b,i} \quad \forall b \in \{1, 2\} \quad i \in \{I, II\} \quad (\text{subproblem formulation})
\end{aligned}$$

So now, we assign lambda values to the vertices of the feasible polytope, and since it is a convex polytope, we can get our x values by taking the convex combination of the vertices. As follows,

$$\begin{aligned}
x_{b,I} &= [m_{b,A,I}, m_{b,B,I}] = v_{b,1,I}\lambda_{b,1,I} + v_{b,2,I}\lambda_{b,2,I} + v_{b,3,I}\lambda_{b,3,I} \\
&= [0, m_{b,I}]\lambda_{b,1,I} + [m_{b,I}, 0]\lambda_{b,2,I} + [0, 0]\lambda_{b,3,I} \\
&= [0, \lambda_{b,1,I}m_{b,I}] + [\lambda_{b,2,I}m_{b,I}, 0] + [0, 0]\lambda_{b,3,I} \\
&= [\lambda_{b,2,I}m_{b,I}, \lambda_{b,1,I}m_{b,I}] + [0, 0]\lambda_{b,3,I}
\end{aligned}$$

So now we can rewrite the problem in terms of the lambda values as the new variables.

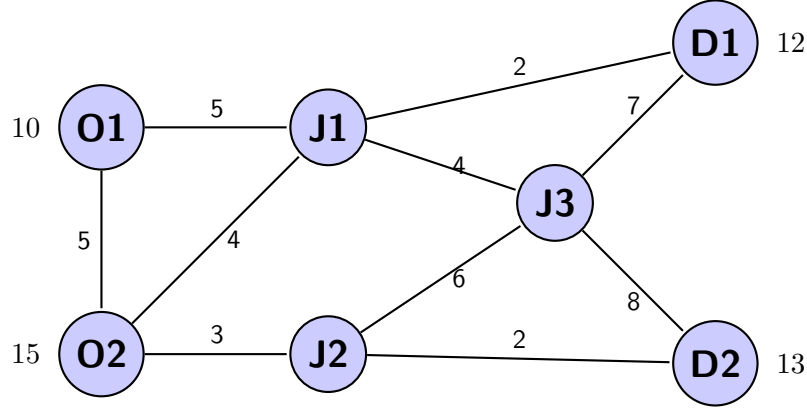
$$\begin{aligned}
& \max[V_A, V_B]([(\lambda_{1,2,Im1,I}, \lambda_{b,1,Im1,I}) + [0, 0]\lambda_{1,3,I})^T + ([\lambda_{1,2,IIIm1,II}, \lambda_{1,1,IIIm1,II}] + [0, 0]\lambda_{1,3,II})^T \\
& \quad + ([\lambda_{2,2,Im2,I}, \lambda_{2,1,Im2,I}] + [0, 0]\lambda_{2,3,I})^T + ([\lambda_{2,2,IIIm2,II}, \lambda_{2,1,IIIm2,II}] + [0, 0]\lambda_{2,3,II})^T) \\
& \text{st. } [\frac{1}{r_{A,I}}, \frac{1}{r_{B,I}}]([(\lambda_{1,2,Im1,I}, \lambda_{1,1,Im1,I}) + [0, 0]\lambda_{1,3,I})^T + ([\lambda_{2,2,Im2,I}, \lambda_{2,1,Im2,I}] + [0, 0]\lambda_{2,3,I})^T) \\
& \quad + [\frac{1}{r_{A,II}}, \frac{1}{r_{B,II}}]([(\lambda_{1,2,IIIm1,II}, \lambda_{1,1,IIIm1,II}) + [0, 0]\lambda_{1,3,II})^T \\
& \quad + ([\lambda_{2,2,IIIm2,II}, \lambda_{2,1,IIIm2,II}] + [0, 0]\lambda_{2,3,II})^T) \leq t_{\text{period}} \quad (1) \\
& ([r_{A,I}, 0]([\lambda_{1,2,Im1,I}, \lambda_{1,1,Im1,I}] + [0, 0]\lambda_{1,3,I})^T + [r_{A,I}, 0]([\lambda_{2,2,Im2,I}, \lambda_{2,1,Im2,I}] + [0, 0]\lambda_{2,3,I})^T) \\
& \quad - ([r_{A,II}, 0]([\lambda_{1,2,IIIm1,II}, \lambda_{1,1,IIIm1,II}] + [0, 0]\lambda_{1,3,II})^T + [r_{A,II}, 0]([\lambda_{2,2,IIIm2,II}, \lambda_{2,1,IIIm2,II}] \\
& \quad + [0, 0]\lambda_{2,3,II})^T) = 0 \\
& ([0, r_{B,I}]([\lambda_{1,2,Im1,I}, \lambda_{1,1,Im1,I}] + [0, 0]\lambda_{1,3,I})^T + [0, r_{B,I}]([\lambda_{2,2,Im2,I}, \lambda_{2,1,Im2,I}] + [0, 0]\lambda_{2,3,I})^T) \\
& \quad - ([0, r_{B,II}]([\lambda_{1,2,IIIm1,II}, \lambda_{1,1,IIIm1,II}] + [0, 0]\lambda_{1,3,II})^T + [0, r_{B,II}]([\lambda_{2,2,IIIm2,II}, \lambda_{2,1,IIIm2,II}] \\
& \quad + [0, 0]\lambda_{2,3,II})^T) = 0 \\
& \sum_{j \leq 3} \lambda_{b,j,i} = 1 \quad \forall b \in \{1, 2\} \quad i \in \{I, II\} \\
& \lambda_{b,j,i} \geq 0 \quad \forall b \in \{1, 2\} \quad j \in \{1, 2, 3\} \quad i \in \{I, II\}
\end{aligned}$$

$$\begin{aligned}
& \max[V_A\lambda_{1,2,Im1,I}, V_B\lambda_{1,1,Im1,I}] + [V_A\lambda_{1,2,IIIm1,II}, V_B\lambda_{1,1,IIIm1,II}] \\
& \quad + [V_A\lambda_{2,2,Im2,I}, V_B\lambda_{2,1,Im2,I}] + [V_A\lambda_{2,2,IIIm2,II}, V_B\lambda_{2,1,IIIm2,II}] \\
& \text{st. } [\frac{1}{r_{A,I}}, \frac{1}{r_{B,I}}]([(\lambda_{1,2,Im1,I}, \lambda_{1,1,Im1,I})^T + ([\lambda_{2,2,Im2,I}, \lambda_{2,1,Im2,I}])^T) \\
& \quad + [\frac{1}{r_{A,II}}, \frac{1}{r_{B,II}}]([(\lambda_{1,2,IIIm1,II}, \lambda_{1,1,IIIm1,II})^T + ([\lambda_{2,2,IIIm2,II}, \lambda_{2,1,IIIm2,II}])^T) \\
& \quad + t_{\text{idle}} = t_{\text{period}} \quad (1) \\
& (r_{A,I}\lambda_{1,2,Im1,I} + r_{A,I}\lambda_{2,2,Im2,I}) - (r_{A,II}\lambda_{1,2,IIIm1,II} + r_{A,II}\lambda_{2,2,IIIm2,II}) = 0 \\
& (r_{B,I}\lambda_{1,1,Im1,I} + r_{B,I}\lambda_{2,1,Im2,I}) - (r_{B,II}\lambda_{1,1,IIIm1,II} + r_{B,II}\lambda_{2,1,IIIm2,II}) = 0 \\
& \sum_{j \leq 3} \lambda_{b,j,i} = 1 \quad \forall b \in \{1, 2\} \quad i \in \{I, II\} \\
& t_{\text{idle}}, \lambda_{b,j,i} \geq 0 \quad \forall b \in \{1, 2\} \quad j \in \{1, 2, 3\} \quad i \in \{I, II\}
\end{aligned}$$

So this is the Master problem, now we proceed to create an RMP from a subset of variables in our master problem, it is to be noted that we can't only take pairs of one of the lambdas from a subproblem otherwise, we would create a problem that is not feasible. Hence, we should always include the zero lambdas for all the blocks that we choose to include in the RMP.

Now consider the RMP with the variables, $\lambda_{1,2,I}, \lambda_{1,3,I}, \lambda_{2,2,I}, \lambda_{2,1,I}, \lambda_{1,3,I}, \lambda_{2,3,I}$.

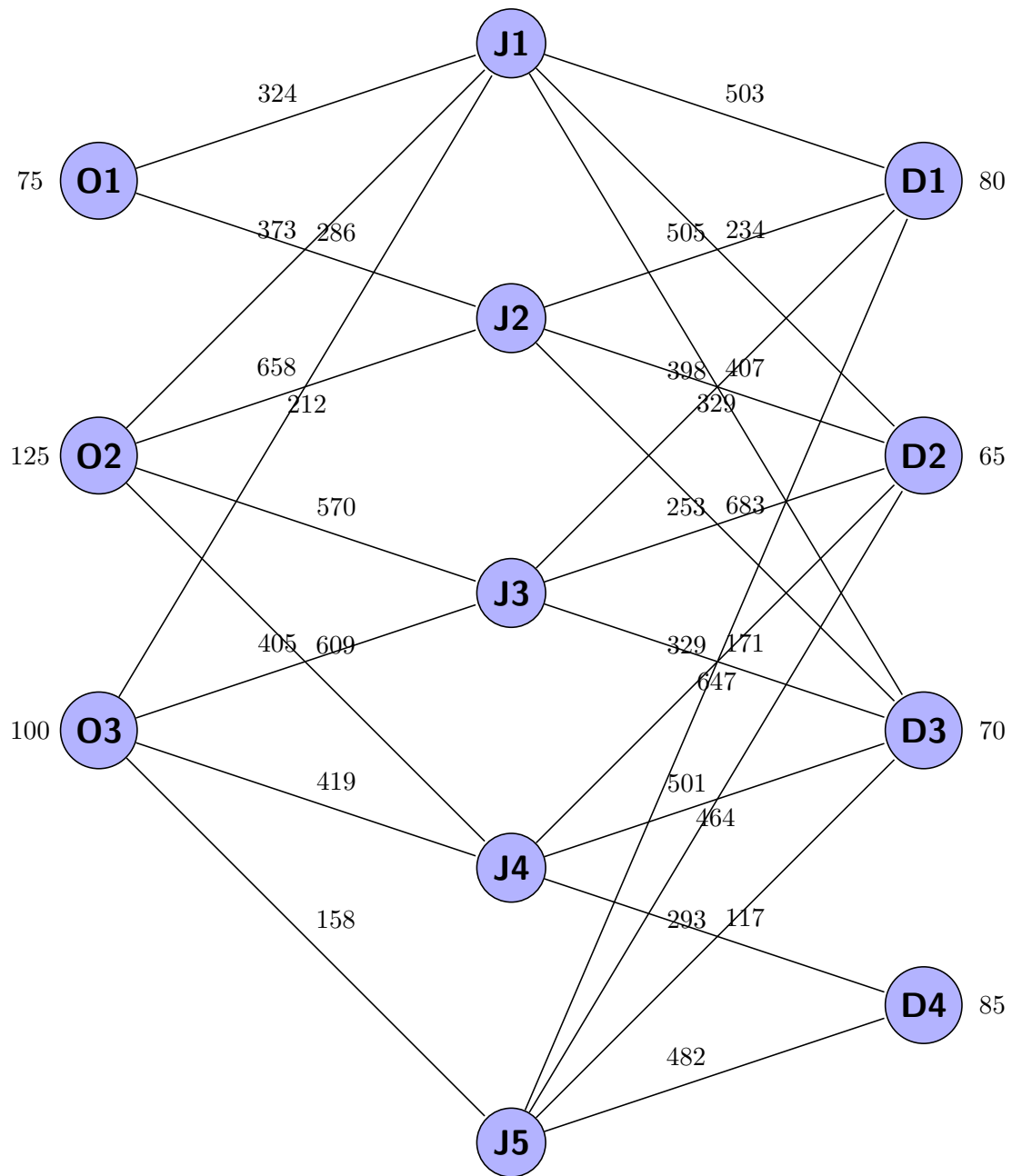
1. Transshipment Problem



We can formulate this problem as a linear program in the following way,

$$\begin{aligned}
 \min \quad & \sum_{u,v \in V} t_{u,v} x_{u,v} \\
 \text{s.t.} \quad & \sum_{v \in V} x_{u,v} - \sum_{v \in V} x_{v,u} = a_u \quad \forall u \in V \\
 & \sum_{v \in V} x_{v,u} - \sum_{v \in V} x_{u,v} = b_u \quad \forall u \in V \\
 & \sum_{i=1}^m a_i = \sum_{j=1}^n b_j \\
 & x_{u,v} \geq 0 \quad \forall u, v \in V
 \end{aligned}$$

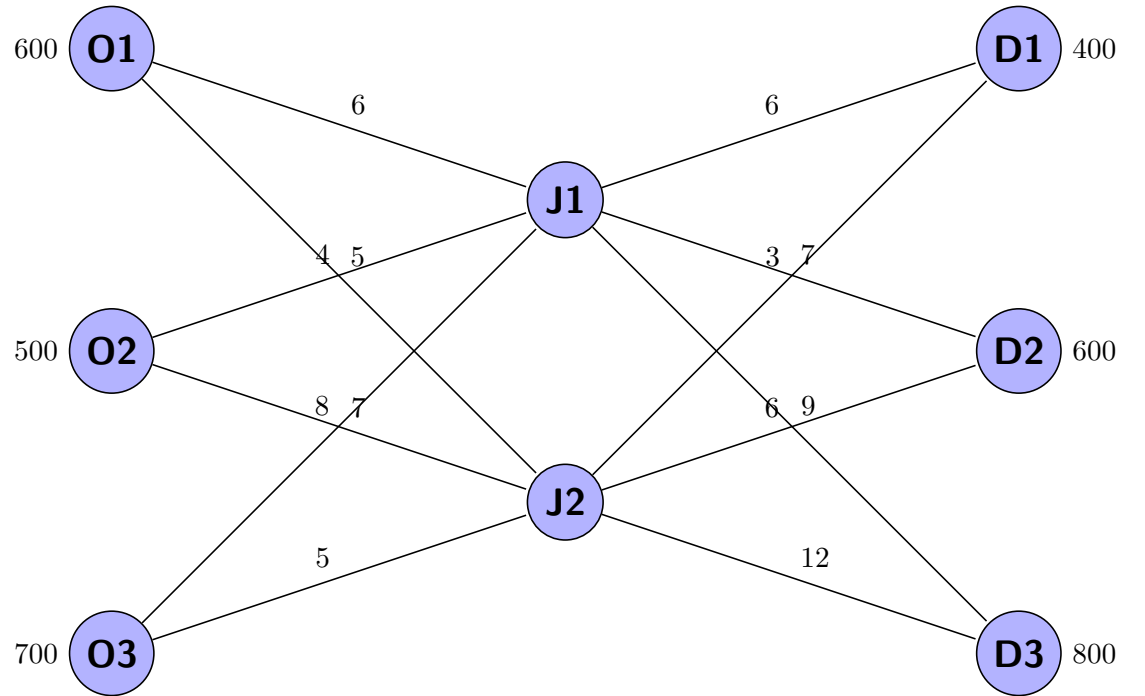
consider the following example,



Additional example.

	WH 1	WH 2	Capacity
Plant 1	6	5	600
Plant 2	4	7	500
Plant 3	8	5	700

	DC 1	DC 2	DC 3
WH 1	6	7	9
WH 2	3	6	12
Demand	400	600	800



$$\begin{aligned}
& \text{s.t.} \quad x_{o_1,j_1} + x_{o_1,j_2} \leq 600 \\
& x_{o_2,j_1} + x_{o_2,j_2} \leq 500 \\
& x_{o_3,j_1} + x_{o_3,j_2} \leq 700 \\
& x_{o_1,j_1} + x_{o_2,j_1} + x_{o_3,j_1} - x_{j_1,d_1} - x_{j_1,d_2} - x_{j_1,d_3} = 0 \\
& x_{o_1,j_2} + x_{o_2,j_2} + x_{o_3,j_2} - x_{j_2,d_1} - x_{j_2,d_2} - x_{j_2,d_3} = 0
\end{aligned}$$

$$\begin{aligned}
x_{j_1,d_1} + x_{j_2,d_1} &= 400 \\
x_{j_1,d_2} + x_{j_2,d_2} &= 600 \\
x_{j_1,d_3} + x_{j_2,d_3} &= 800 \\
x_{u,v} &\geq 0 \quad \forall u, v \in V
\end{aligned}$$

2. Multiple Vehicle Routing

$$\begin{aligned}
\min \quad & \sum_{u \in V} \sum_{v \in V} w_{u,v} x_{u,v} \\
\text{s.t.} \quad & \sum_{v \in V} x_{v,u} = 1 \quad \forall u \in V \setminus \{0\} \\
& \sum_{u \in V} x_{v,u} = 1 \quad \forall v \in V \setminus \{0\} \\
& \sum_{v \in V \setminus \{0\}} x_{u,0} = \sum_{u \in V \setminus \{0\}} x_{0,v} = k \\
& \sum_{v \notin S} \sum_{u \in S} x_{v,u} \geq r(S), \quad \forall S \subseteq V \setminus \{0\}, S \neq \emptyset \\
& x_{v,u} \in \{0, 1\} \quad \forall v, u \in V
\end{aligned}$$

The 4th constraint can be rewritten as follows,

$$\sum_{v \in S} \sum_{u \in S} x_{v,u} \leq |S| - r(S)$$

3. Multidivisional problem

$$\begin{aligned}
\max \quad & 8x_1 + 5x_2 + 6x_3 + 9x_4 + 7x_5 + 9x_6 + 6x_7 + 5x_8 + 6x_9 \\
\text{s.t.} \quad & 5x_1 + 3x_2 + 4x_3 + 2x_4 + 7x_5 + 3x_6 + 4x_7 + 6x_8 + x_9 \leq 30 \\
& 2x_1 + 4x_2 + 3x_3 \leq 5 \\
& 2x_4 + 8x_5 + 6x_6 \leq 6 \\
& 3x_7 + 5x_8 + 9x_9 \leq 32 \\
& x_i \geq 0 \quad \forall i \in \{1, \dots, 9\} \quad \text{and} \quad x_i \in \mathbb{Z} \quad \forall i \in \{1, \dots, 9\}
\end{aligned}$$

We then do the benders decomposition, with initial master problem,

$$\begin{array}{ll}
\max & 8x_1 + 5x_2 + 6x_3 + 9x_4 + 7x_5 + 9x_6 + 6x_7 + 5x_8 + 6x_9 \\
\text{s.t.} & 5x_1 + 3x_2 + 4x_3 + 2x_4 + 7x_5 + 3x_6 + 4x_7 + 6x_8 + x_9 \leq 30
\end{array}$$

and the following subproblems,

$$\begin{array}{ll}
\max & \pi_1 (5x_1 + 3x_2 + 4x_3) + \pi_2 (2x_1 + 4x_2 + 3x_3) \\
\text{s.t.} & 2x_1 + 4x_2 + 3x_3 \leq 5 \\
& x_1, x_2, x_3 \geq 0, \quad x_1, x_2, x_3 \in \mathbb{Z}
\end{array}$$

$$\begin{array}{ll}
\max & \pi_3 (2x_4 + 8x_5 + 6x_6) + \pi_4 (2x_4 + 7x_5 + 3x_6) \\
\text{s.t.} & 2x_4 + 8x_5 + 6x_6 \leq 6 \\
& x_4, x_5, x_6 \geq 0, \quad x_4, x_5, x_6 \in \mathbb{Z}
\end{array}$$

$$\begin{array}{ll}
\max & \pi_5 (3x_7 + 5x_8 + 9x_9) + \pi_6 (4x_7 + 6x_8 + x_9) \\
\text{s.t.} & 3x_7 + 5x_8 + 9x_9 \leq 32 \\
& x_7, x_8, x_9 \geq 0, \quad x_7, x_8, x_9 \in \mathbb{Z}
\end{array}$$

After solving the subproblems, we get the following solutions,

$$\begin{array}{ll}
\max & 8x_1 + 5x_2 + 6x_3 + 9x_4 + 7x_5 + 9x_6 + 6x_7 + 5x_8 + 6x_9 \\
\text{s.t.} & 5x_1 + 3x_2 + 4x_3 + 2x_4 + 7x_5 + 3x_6 + 4x_7 + 6x_8 + x_9 \leq 30 \\
& 2x_1 \leq 16 \\
& 3x_4 \leq 27 \\
& 10x_7 \leq 60 \\
& x_i \geq 0 \quad \forall i \in \{1, \dots, 9\} \quad \text{and} \quad x_i \in \mathbb{Z} \quad \forall i \in \{1, \dots, 9\}
\end{array}$$

1. Block-Angular Structure

A matrix A has a **block angular structure** if it is of this form:

$$A = \begin{bmatrix} A_1 & A_2 & \cdots & A_{n-1} & A_n \\ A_{n+1} & 0 & \cdots & 0 & 0 \\ 0 & A_{n+2} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & A_{2n} \end{bmatrix}$$

where A_i are submatrices of size $m_i \times n_i$. We call the first n matrices $A_1, A_2, \dots, A_{n-1}$ the bounding constraints and the last n matrices the independent constraints. Notice that this easily becomes a linear program that looks like this,

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n \vec{c}_i \vec{x}_i \\ & \text{subject to} && [A_1, \dots, A_n, I] \vec{x}^T = \vec{b}_0 \quad \text{linking constraints} \\ & && A_{n+i} \vec{x}_i = \vec{b}_i, \quad \forall i \in \{1, \dots, n\} \quad \text{independent constraints} \\ & && \vec{x}^T \geq 0, \quad \vec{x}_i \geq 0, \quad \forall i \in \{1, \dots, n\} \quad \text{non-negativity constraints} \end{aligned}$$

In this linear program, the $\vec{c}_i$ are the costs and $\vec{x} = [\vec{x}_1, \dots, \vec{x}_n]^T$ are the decision variables and $\vec{b}_i$ are the right hand side of the constraints. The aim of the Dantzig-Wolfe decomposition is to be more computationally efficient in solving this problem. We start with a **restricted master problem** (RMP), which usually encompasses all the linking constraints and then check the independent constraints one by one by solving them as subproblems.

2. Column Generation

2.1 Intro

2.2 Formulating the problem

We denote the optimal solutions as x^* and y^* (by duality, $c^T x^* = b^T y^*$, we denote the optimal value as z^*). We can consider the optimal value as a function of the parameters c, b, A , i.e. $z^* = f(c, b, A)$. We can also consider the optimal dual value of a constraint, y_i^* as a partial derivative of the optimal value with respect to the constraint, i.e. $y_i^* = \frac{\partial z^*}{\partial b_i}$ and further, $y^* = \frac{\partial z^*}{\partial b}$. Notably, y_i^* represents the increase in the optimal value of the objective function if the RHS of the i th constraint is increased by 1.

2.3 Altered Simplex Method

2.4 Computational Analysis

2.5 Examples

3. Dantzig-Wolfe Decomposition

3.1 Overview

We'll start by giving a high level view of the algorithm, which is basically doing these steps (Wikipedia):

1. Start with a feasible solution to the RMP, formulate new objective functions for each subproblem such that the subproblems will offer solutions that improve the current objective of the master program.
2. Re-solve the subproblems given their new objective functions, the optimal value for each subproblem is offered to the master program.
3. Incorporate one or all of the new columns generated by the solutions to the subproblems into the master program based on their respective ability to improve the original problem's objective.
4. Perform x iterations of the simplex algorithm in the master program, where x is the number of columns incorporated.
5. If the objective is improved, go to step 1. Otherwise, we ran out of improvement so we return the solution.

Note that Dantzig-Wolfe is most effective when the subproblems are easy to solve. Now, let's break it down into more detail. The first step consists of solving the RMP and then defining the objective functions. The RMP looks like this,

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n \vec{c}_i \vec{x}_i \\ & \text{subject to} && [A_1, \dots, A_n, I] \vec{x}^T = \vec{b}_0 \\ & && \vec{x}^T \geq 0 \end{aligned}$$

The subproblems look like this, for every i th subproblem,

$$\begin{aligned} & \text{maximize} && \vec{c}_i \vec{x}_i \\ & \text{subject to} && A_{n+i} \vec{x}_i = \vec{b}_i \\ & && \vec{x}_i \geq 0 \end{aligned}$$

What interests us is the feasible region of the subproblems, we will denote them as $S_i = \{\vec{x}_i \geq 0 | A_{n+i}\vec{x}_i = \vec{b}_i\}$. Assume that S_i is bounded for simplicity (it can easily be extended to having unbounded regions, we would consider unbounded rays). We have a set of vertices $V_i = \{(v_{i,1}, \dots, v_{i,n}) | v_{i,j} \in S_i\}$ which completely defines the feasible region of the subproblem. This implies that we can represent every $x_i \in S_i$ as a convex combination of the vertices $v_{i,j}$, i.e. $\exists \{\lambda_{i,j} \geq 0\}_j$ s.t. $x_i = \sum_{j=1}^n \lambda_{i,j} v_{i,j}$ and with $\sum_{j=1}^n \lambda_{i,j} = 1$ (this follows from the representation theorem).

Now, we can transform the RMP by using the $\lambda_{i,j}$ as the new decision variables.

$$\begin{aligned} \min \quad & \sum_{i=1}^m \sum_{j=1}^{N_i} \lambda_{i,j} (c_i v_{i,j}) \\ \text{s.t.} \quad & \sum_{i=1}^m \sum_{j=1}^{N_i} \lambda_{i,j} (A_{0,i} v_{i,j}) = b_0 \\ & \sum_{j=1}^{N_i} \lambda_{i,j} = 1 \quad \forall i = 1, 2, \dots, m \\ & \lambda_{i,j} \geq 0 \quad \forall i, j \end{aligned}$$

Notably, we reduced the number of constraints to $m + m_0$, since there aren't now m independent constraints and m_0 linking constraints. Basically, we squashed all the independent constraint blocks into single constraints. However, there are now substantially more variables, which is why we will use an altered simplex method.

3.2 Altered Simplex

We start by creating the dual variables, $y_0 \in \mathbb{R}^{m_0}$ and $z \in \mathbb{R}^m$ which set the sum $\sum_{i=1}^m \lambda_{i,j} = 1$. Note that each column represents one vertex. And so, we want to find out which columns have negative reduced cost (because this is a minimization problem). If we consider the column for $\lambda_{i,j}$, then the reduced cost is

$$\bar{c}_i = c_i v_{i,j} - y_0 A_{0,i} v_{i,j} - z_i$$

which we can obtain by taking the inner product of the dual variables with the column for $\lambda_{i,j}$.

$$\begin{bmatrix} y_0 & z \end{bmatrix} \begin{bmatrix} A_{0,i} v_{i,j} \\ c_i \end{bmatrix} \quad (\text{unsure about this})$$

So now we want to know if there exists $v_{i,j}$ such that $\bar{c}_i < 0$ or equivalently, $c_i v_{i,j} - y_0 A_{0,i} v_{i,j} - z_i < 0$. If so, we can then find the column for $\lambda_{i,j}$ that

has the smallest reduced cost? This means we want to find a vertex of S_i such that $(c_i - y_0 A_{0,i})v_{i,j} - z_i < 0$.

We now consider the resulting subproblem,

$$\begin{array}{ll} \min & \bar{c}_i x_i \\ \text{s.t.} & A_{n+i} x_i = b_i, \\ & x_i \geq 0 \end{array}$$

Since S_i is nonempty (otherwise the master problem is infeasible), this problem must contain a vertex $\mathbf{v}$ which is the optimal solution. If $\bar{c}_i \mathbf{v} < z_i$ then we know that $\mathbf{v}$ is a negative cost vertex and we should add it to the basis. If this is false $\forall i$, then no column of S_i has negative reduced cost. Hence, after solving the subproblems for all S_i , we either find a negative reduced cost column or otherwise we show that the current solution to the master problem is optimal.

Notice that this in itself is fundamentally a revised version of the simplex algorithm, however with significantly reduced usage of space. Additionally, it is more efficient computationally since the slave problems can be solved in parallel (that is the big advantage), we simply choose the first negative reduced cost column subproblem to be computed.

3.5 Examples

4. Block Selection Linear Program

4.1 Defining the problem

This is a subproblem of larger mine planning optimization problems. The objective is to maximize the total value of the blocks selected, subject to the constraints that the total volume of the selected blocks does not exceed the available volume, and that the total value of the selected blocks does not exceed the available value. Here is a formal version of the problem for two modes, $\{A, B\}$ and two ore types $\{I, II\}$.

$$\begin{aligned}
& \max \sum_{b \in \mathcal{B}} (V_{b,A,I} m_{b,A,I} + V_{b,A,II} m_{b,A,II} + V_{b,B,I} m_{b,B,I} + V_{b,B,II} m_{b,B,II}) \\
& \text{st. } \sum_{b \in \mathcal{B}} \left(\frac{m_{b,A,I}}{r_{A,I}} + \frac{m_{b,A,II}}{r_{A,II}} + \frac{m_{b,B,I}}{r_{B,I}} + \frac{m_{b,B,II}}{r_{B,II}} \right) \leq t_{\text{period}} \quad (1) \\
& r_{A,I} m_{b,A,I} + r_{A,II} m_{b,A,II} = 0 \quad \forall b \in \mathcal{B} \quad (2) \\
& r_{B,I} m_{b,B,I} + r_{B,II} m_{b,B,II} = 0 \quad \forall b \in \mathcal{B} \quad (3) \\
& m_{b,A,I} + m_{b,B,I} \leq m_{b,I} \\
& m_{b,A,I} + m_{b,B,I} \leq m_{b,I} \quad \forall b \in \mathcal{B} \quad (4) \\
& m_{ijk}, r_{jk} \geq 0 \quad \forall i \in \mathcal{B}, j \in \text{modes}, k \in \text{ores}
\end{aligned}$$

where $\mathcal{B}$ is the set of blocks, V_{bjk} is the value of block b in mode j and ore k , m_{bjk} is the volume of block b in mode j and ore k , r_{jk} is the recovery rate of ore k in mode j , and t_{period} is the total time available for block selection.

The first constraint represents the time constraint, that is to say that the total volume of blocks divided by the rate of recovery of the ore must be less than or equal to the total time available. The second and third constraints represent the volume constraints, that is to say that the volume of blocks selected in mode j and ore k must be less than or equal to the total volume of blocks in mode j and ore k .

4.2 Improvements based on column generation and Dantzig-Wolfe

4.2.1 Column Generation

4.2.2 Dantzig-Wolfe

4.3 Implementation details

4.4 Computational Analysis

4.5 Example with 2 modes and 2 ores

5. Potential Future Work

5. Conclusion

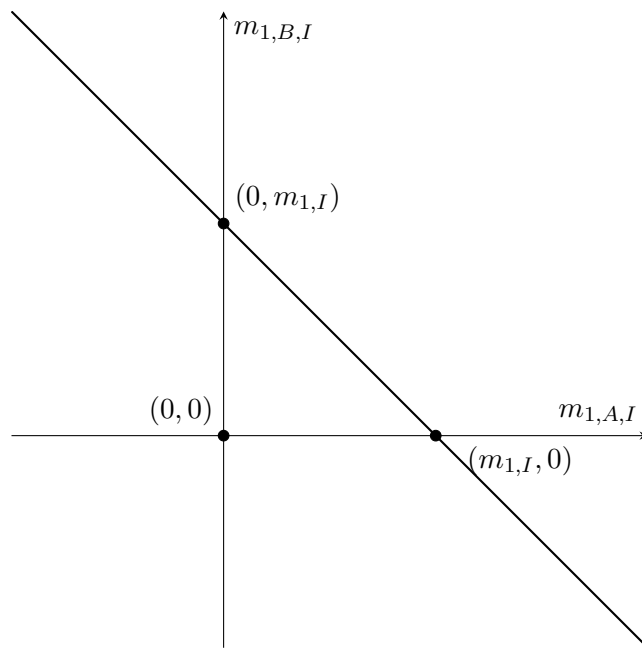


Figure 1: Feasible polytope for the constraint $m_{A,I} + m_{B,I} \leq m_I$